

Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

nihal.altuntas@iuc.edu.tr

Algoritma Analizi ve Big-O Notasyonu

- Bir algoritmanın yalnızca doğru çalışması yeterli midir?
- Daha iyi bir çözüm var mı?
- Daha iyi çözüm ne demek?
  - Hangi çözümün daha iyi olduğuna nasıl karar vereceğiz?

Neden Algoritma Analizi?

- Bir problemi çözmek için birden fazla doğru algoritma olabilir.
- Farklı algoritmalar aynı sonucu üretse de çalışma maliyetleri farklı olabilir.
- Çalışma süresini yalnızca saniye ile ölçmek; bilgisayar, dil ve sistem özelliklerine bağlıdır.
- Bu nedenle algoritmanın **girdi boyutu** büyüdükçe yaptığı işi inceleriz.

Nasıl?

- n: algoritmanın üzerinde çalıştığı girdi büyüklüğü.
- Bu örnekte n = listedeki eleman sayısı.
- Döngü her eleman için bir kez çalışır.
- n iki katına çıkarsa yapılan iş de yaklaşık iki katına çıkar.

```
numbers = [7, 2, 9, 4, 1, 8]

for x in numbers:
    print(x)
```

Big-O Notasyonu

- Big-O, girdi büyüdükçe algoritmanın çalışma maliyetinin nasıl büyüdüğünü ifade eder.
- Kesin çalışma süresini değil, büyüme davranışını karşılaştırmamızı sağlar.
- Bugün dört temel sınıfa odaklanacağız:
  - O(1) → sabit zaman
  - O(log n) → logaritmik zaman
  - O(n) → doğrusal zaman
  - $O(n^2)$ → karesel zaman

O(1) — Sabit Zaman

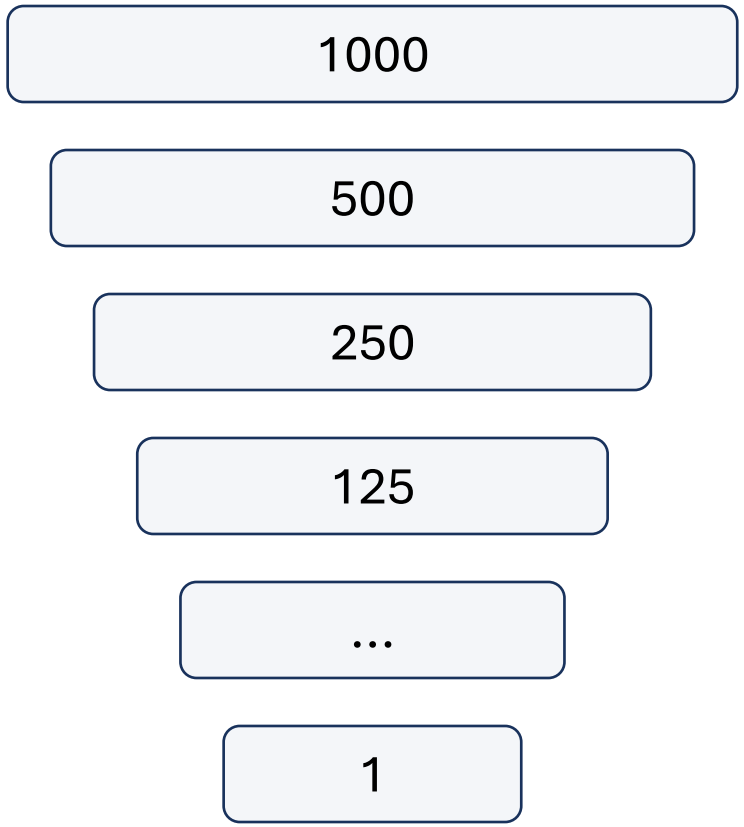
- Tek bir indeks üzerinden elemana erişim gerçekleştirilir.
- Liste 5 elemanlı da olsa, 5 milyon elemanlı da olsa aynı tür işlem yapılır.
- İşlem sayısı n ile büyümez → O(1)

```
numbers = [10, 20, 30, 40, 50]

print(numbers[2])
```

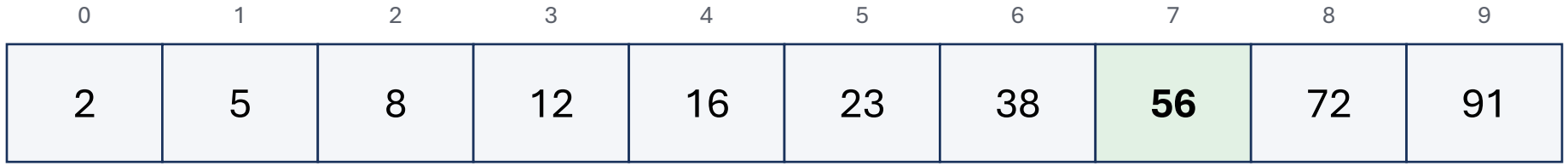
O(log n) — Logaritmik Zaman

- Her adımda problem boyutu belirli bir oranda düşürülür
  - En tipik örnek: sıralı bir dizide binary search (ikili arama).
  - Her karşılaştırmadan sonra arama alanının yaklaşık yarısı elenir.
  - 10 eleman için birkaç adım; 1000 eleman için yaklaşık 10 adım yeterlidir.
  - n iki katına çıktığında işlem sayısı yalnızca yaklaşık 1 artar → O(log n)



Linear Search ve Binary Search Karşılaştırması

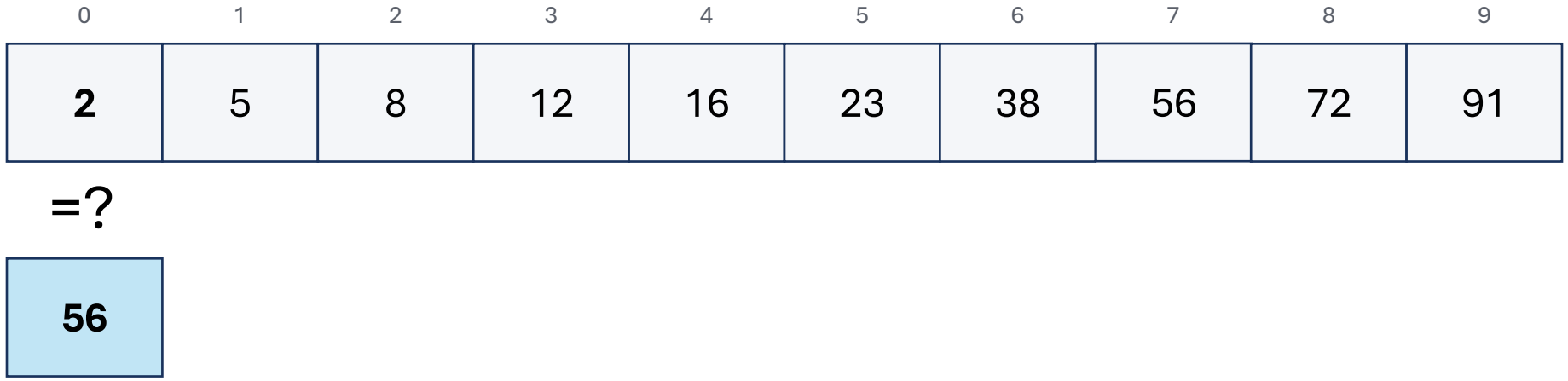
- Aşağıda verilen sıralı dizide bir sayısının indeksini bulma
  - Aranan sayı: 56



Bu örnekte
8 karşılaştırma ↔ 2 karşılaştırma

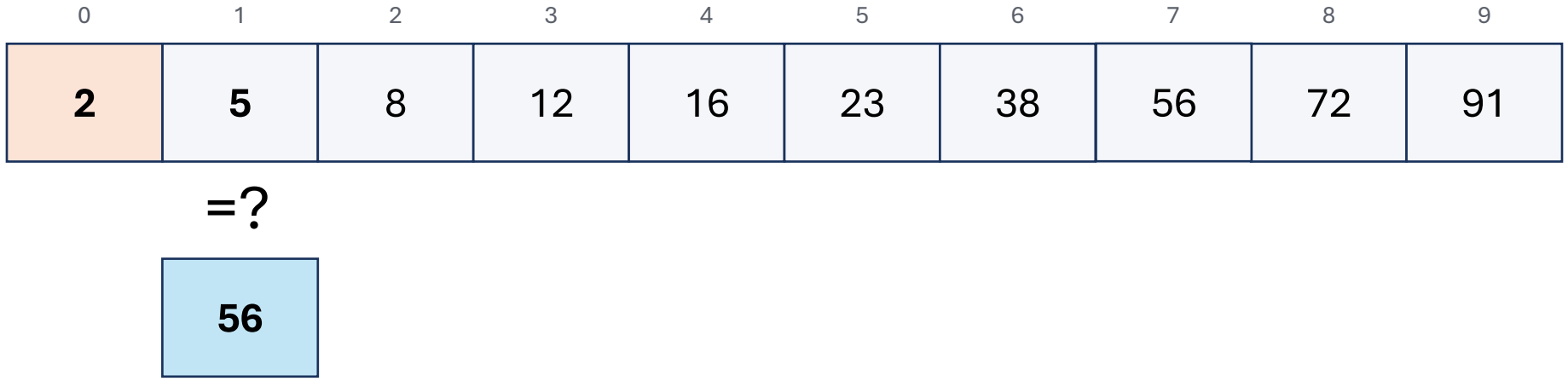
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



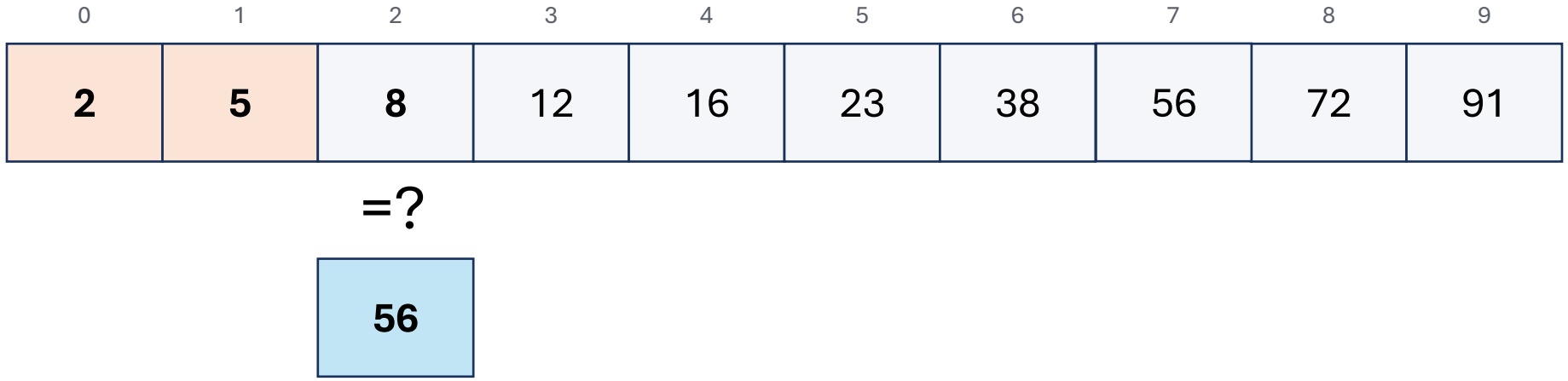
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



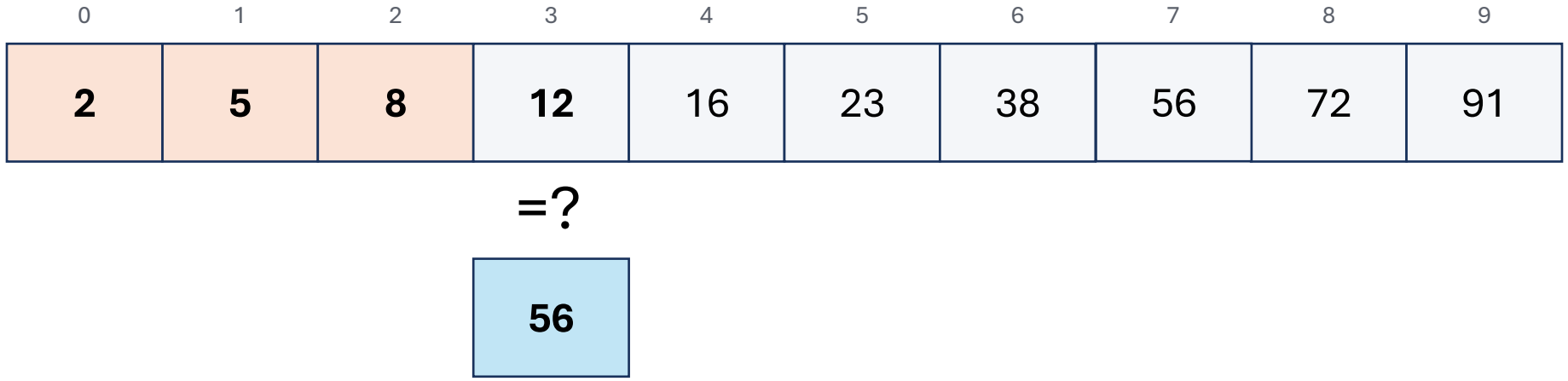
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



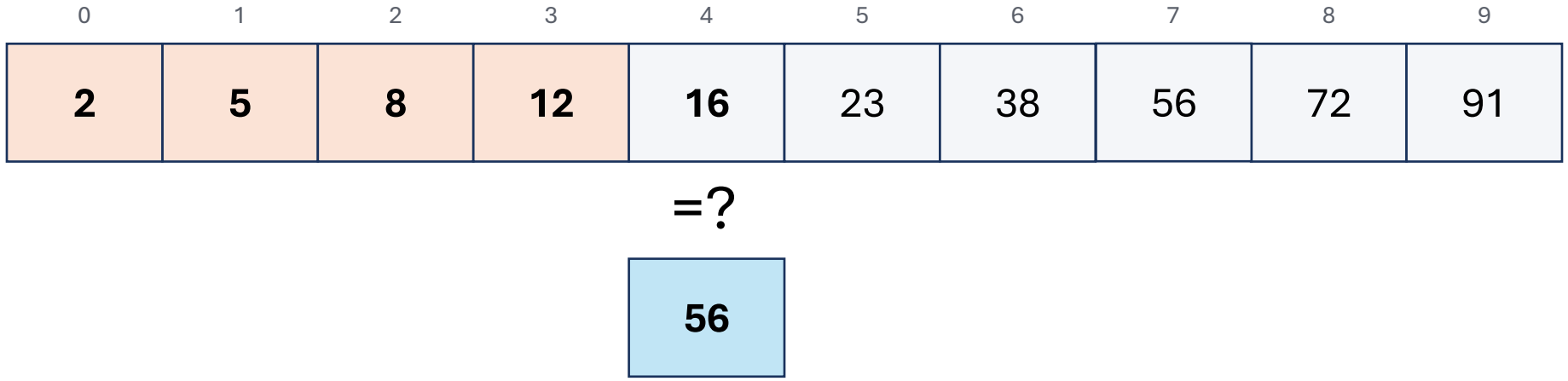
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



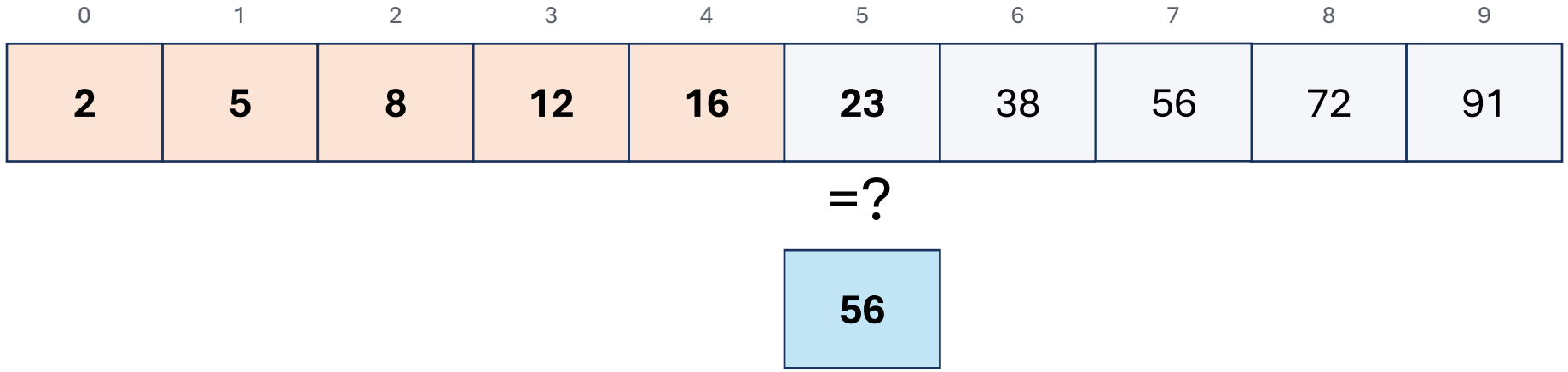
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



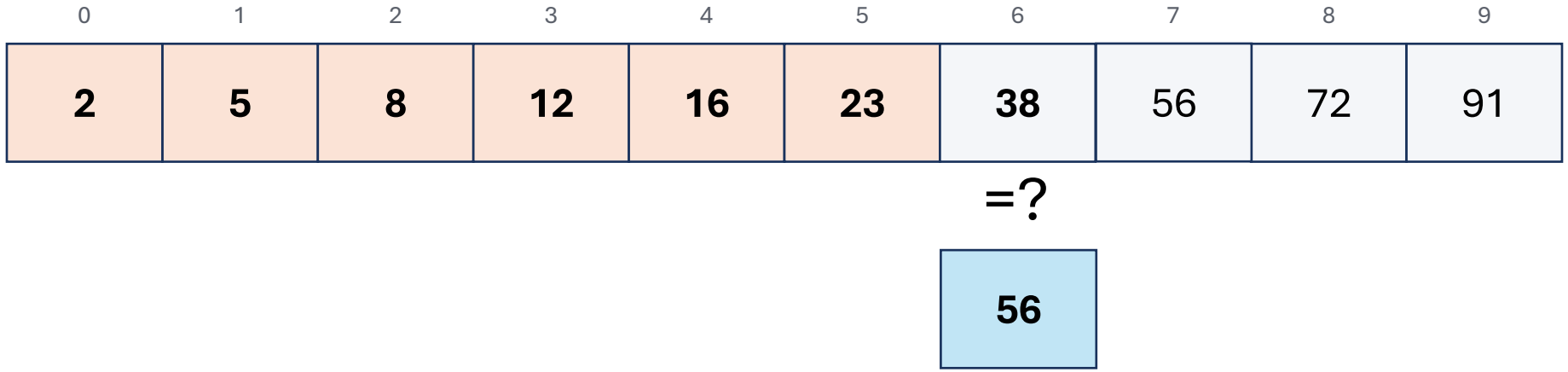
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



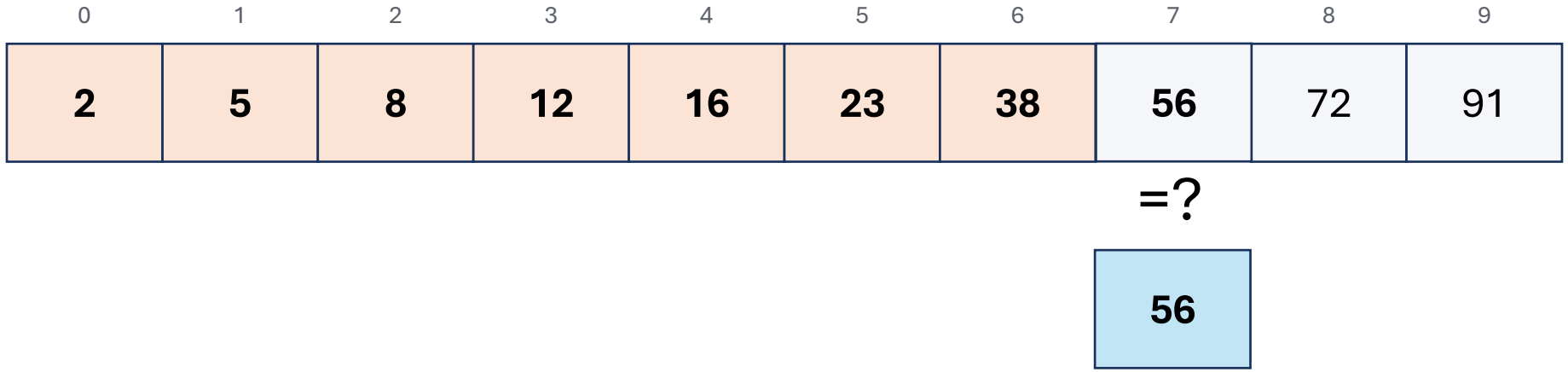
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



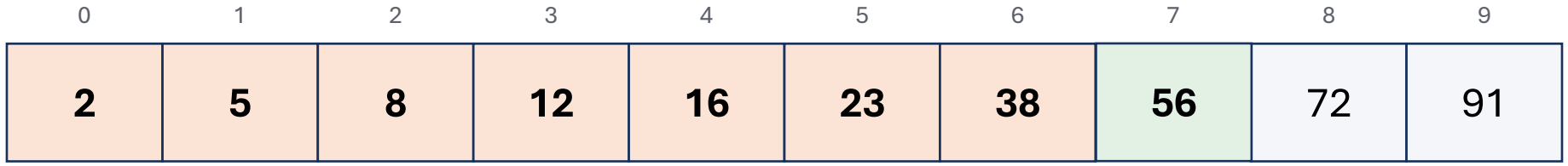
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



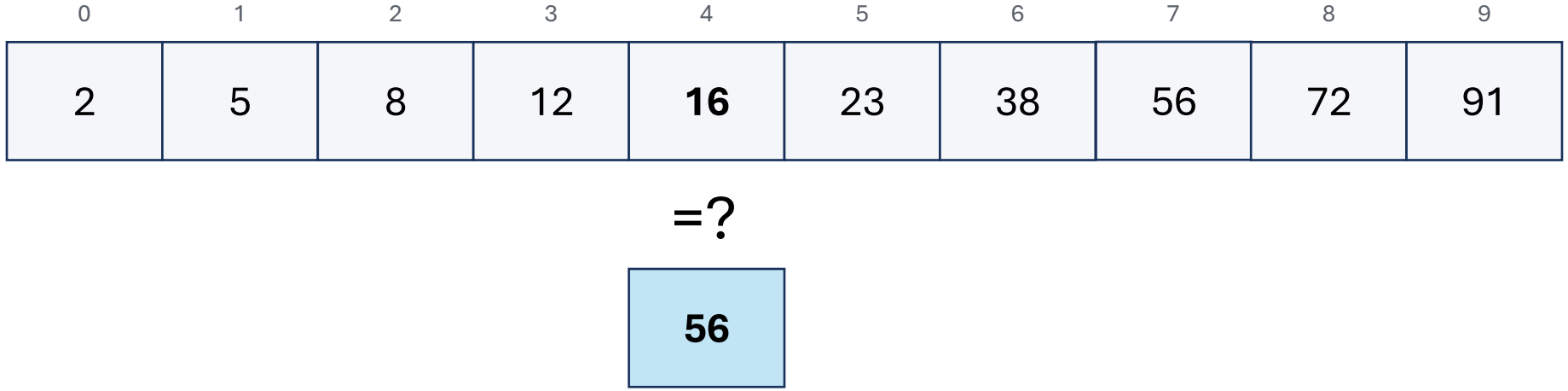
Linear Search ve Binary Search Karşılaştırması

- Linear Search — O(n)
  - Soldan başlar: 2 → 5 → 8 → 12 → 16 → 23 → 38 → 56
  - 56 bulunana kadar 8 karşılaştırma yapılır.
  - En kötü durumda 10 elemanın tamamına bakılır.



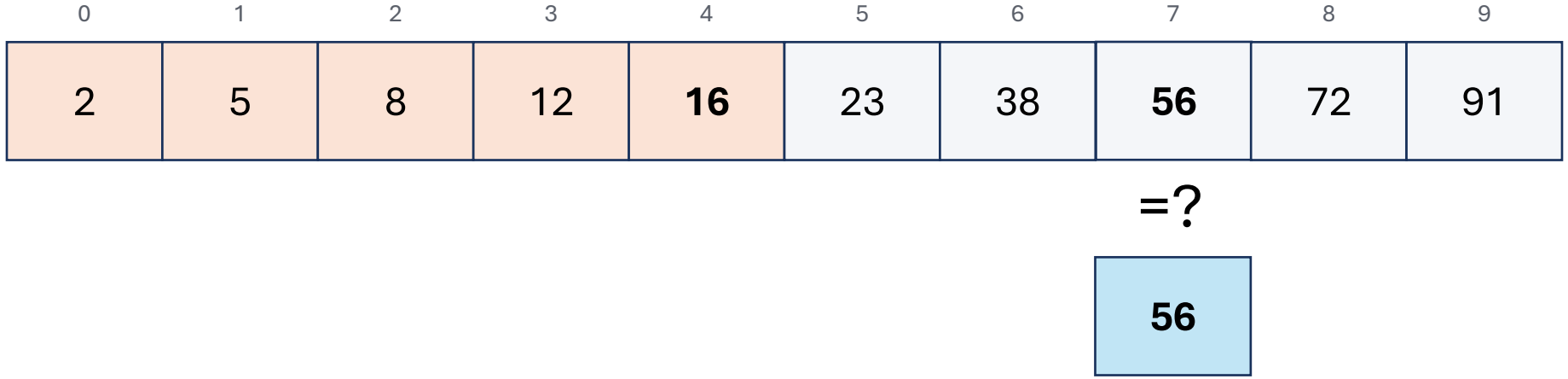
Linear Search ve Binary Search Karşılaştırması

- Binary Search — O(log n)
  - İlk olarak ortadaki elemanla karşılaştırma yapılır: 16
    - Daha büyük olduğu için sol yarı elenir.
  - Sağda kalan bölümün ortasındaki elemanla karşılaştır: 56 → bulundu.
  - Yalnızca 2 karşılaştırma yapılır.



Linear Search ve Binary Search Karşılaştırması

- Binary Search — O(log n)
  - İlk olarak ortadaki elemanla karşılaştırma yapılır: 16
    - Daha büyük olduğu için sol yarı elenir.
  - Sağda kalan bölümün ortasındaki elemanla karşılaştır: 56 → bulundu.
  - Yalnızca 2 karşılaştırma yapılır.



Linear Search ve Binary Search Karşılaştırması

- Binary Search — O(log n)
  - İlk olarak ortadaki elemanla karşılaştırma yapılır: 16
    - Daha büyük olduğu için sol yarı elenir.
  - Sağda kalan bölümün ortasındaki elemanla karşılaştır: 56 → bulundu.
  - Yalnızca 2 karşılaştırma yapılır.



O(n) — Doğrusal Zaman

- Soru: Aradığımız değer en sonda ise kaç elemana bakmamız gerekir?
  - En kötü durumda bütün elemanları kontrol ederiz.
  - n eleman varsa en fazla n kontrol gerekir.
  - n iki katına çıktığında iş miktarı da yaklaşık iki katına çıkar → O(n)

```
target = 8

for x in numbers:
    if x == target:
        print("Bulundu")
        break
```

$O(n^2)$ — Karesel Zaman

- Soru: İçteki print toplam kaç kez çalışır?
  - Dış döngü n kez çalışır.
  - Her dış döngü adımında iç döngü n kez çalışır.
  - Toplam yaklaşık $n \times n = n^2$ işlem.
  - n iki katına çıkarsa iş yaklaşık dört katına çıkar → $O(n^2)$

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Hangisi Daha Hızlı Büyür?

- Küçük girdilerde fark az görünebilir; n büyüdükçe algoritma seçimi kritik hale gelir.

n	O(1)	$O(\log_2 n)$	O(n)	$O(n\log_2 n)$	$O(n^2)$
1	1	1	1	1	1
10	1	≈ 3.3	10	≈ 33	100
100	1	≈ 6.6	100	≈ 664	10 000
1000	1	≈ 10	1 000	≈ 9 966	1 000 000
10000	1	≈ 13	10 000	≈ 132 877	100 000 000

Veri Yapısı Açısında Algoritma Analizi

- Bir dizide belirli bir değeri bulmak için baştan sona tarama yapmanın maliyet nedir?
- Araya yeni bir eleman eklemek için sağdaki elemanları kaydırmak gerekirse maliyet nedir?
- Bir elemanı silip boşluğu kapatmak için elemanları sola kaydırınca maliyet nedir?

Algoritma analizi, veri yapısını neden seçtiğimizi anlamamıza yardım eder.